

JavaScript Interview Questions

Objects & Functions — Concepts, Code & Answers

This guide covers the most commonly asked interview questions on JavaScript **objects** and **functions** — from property access and prototypes to closures, `this` binding, higher-order functions, and currying. Each question includes a clear explanation and a short code example.

Contents

1. Object Fundamentals
2. Object Methods & Copying
3. Prototypes & Classes
4. Function Fundamentals
5. `'this'`, `call`/`apply`/`bind` & Arrow Functions
6. Closures, Scope & Higher-Order Functions
7. Coding Challenges

1. Object Fundamentals

Q1. What's the difference between dot notation and bracket notation for accessing properties?

Dot notation (`obj.key`) requires a valid, static identifier name. Bracket notation (`obj['key']`) accepts any string or a dynamic expression, which makes it necessary for computed or variable property names.

```
const obj = { 'first-name': 'Sam', age: 30 };
obj['first-name'];    // 'Sam' -- dot notation would fail here

const key = 'age';
obj[key];             // 30 -- dynamic access
```

Q2. How do you check if a property exists on an object?

Use `'key' in obj` (checks the prototype chain too), `obj.hasOwnProperty('key')` (own properties only), or simply check `obj.key !== undefined` — though that last approach fails if the value is legitimately undefined.

```
const obj = { a: undefined, b: 1 };
'a' in obj;           // true
obj.hasOwnProperty('a'); // true
obj.a !== undefined;  // false -- misleading!
```

Q3. What's the difference between null and undefined?

`undefined` means a variable has been declared but not assigned a value (JavaScript's default). `null` is an intentional assignment representing "no value." They are loosely equal (`null == undefined`) but not strictly equal.

```
let a;           // undefined
let b = null;    // null

a == b;  // true  (loose equality)
a === b; // false (different types)
```

Q4. How do you iterate over the keys, values, or entries of an object?

`Object.keys()`, `Object.values()`, and `Object.entries()` return arrays you can loop over, or you can use `for...in` (which also walks inherited enumerable properties, unlike the `Object.*` methods).

```
const user = { name: 'Ana', age: 28 };

Object.keys(user);    // ['name', 'age']
Object.values(user);  // ['Ana', 28]
Object.entries(user); // [['name', 'Ana'], ['age', 28]]

for (const [k, v] of Object.entries(user)) {
  console.log(`${k}: ${v}`);
}
```

2. Object Methods & Copying

Q5. What is the difference between a shallow copy and a deep copy of an object?

A shallow copy duplicates only the top-level properties — nested objects are still shared by reference. A deep copy recursively duplicates every nested object, so the copy is fully independent of the original.

```
const original = { name: 'Kit', address: { city: 'NYC' } };

const shallow = { ...original };
shallow.address.city = 'LA';
original.address.city; // 'LA' -- nested object was shared!

const deep = structuredClone(original); // true deep copy
```

Q6. What does `Object.freeze()` do, and how is it different from `Object.seal()`?

`Object.freeze()` prevents adding, removing, *and* modifying properties — the object becomes fully immutable (shallowly). `Object.seal()` prevents adding or removing properties, but existing ones can still be reassigned.

```
const frozen = Object.freeze({ x: 1 });
frozen.x = 2; // silently fails (throws in strict mode)

const sealed = Object.seal({ x: 1 });
sealed.x = 2; // works -- sealed only blocks add/remove
```

Q7. How do you merge two or more objects together?

Use `Object.assign(target, ...sources)` or the spread operator `{...obj1, ...obj2}`. Both perform a shallow merge; properties in later sources overwrite earlier ones with the same key.

```
const defaults = { theme: 'light', fontSize: 14 };
const userPrefs = { fontSize: 18 };

const merged = { ...defaults, ...userPrefs };
// { theme: 'light', fontSize: 18 }
```

Q8. What are getters and setters, and why use them?

Getters and setters let you define computed or validated access to a property while it still looks like a plain property from the outside — useful for derived values or enforcing invariants without changing the calling code.

```
const circle = {
  radius: 5,
  get area() { return Math.PI * this.radius ** 2; },
  set diameter(d) { this.radius = d / 2; }
};

circle.area; // 78.53...
circle.diameter = 20;
circle.radius; // 10
```

3. Prototypes & Classes

Q9. Explain prototypal inheritance in JavaScript.

Every object has an internal link ([[Prototype]]) to another object. When a property isn't found on the object itself, JavaScript looks up the prototype chain until it finds the property or reaches null. This is how objects "inherit" methods.

```
const animal = { speak() { return 'Some sound'; } };
const dog = Object.create(animal);
dog.speak(); // 'Some sound' -- found via the prototype chain

Object.getPrototypeOf(dog) === animal; // true
```

Q10. How do ES6 classes relate to prototypes?

Classes are largely syntactic sugar over JavaScript's existing prototype-based inheritance — methods defined in a class body are added to the constructor's prototype object, not copied onto each instance.

```
class Animal {
  constructor(name) { this.name = name; }
  speak() { return `${this.name} makes a sound.`; }
}

typeof Animal; // 'function'
Animal.prototype.speak === Animal.prototype.speak; // true
```

Q11. What's the difference between an instance property and a static property?

Instance properties/methods belong to each object created from the class and are accessed via an instance. static members belong to the class itself and are called directly on the class, often used for factory methods or utilities.

```
class Circle {
  constructor(r) { this.r = r; }
  area() { return Math.PI * this.r ** 2; }
  static unitCircle() { return new Circle(1); }
}

Circle.unitCircle().area(); // called on the class, not an instance
```

Q12. What is the difference between __proto__ and prototype?

prototype is a property that exists only on functions/classes — it's the object that becomes the prototype of instances created with new. __proto__ is the actual internal link an object uses to look up inherited properties, available on nearly every object.

```
function Dog() {}
const rex = new Dog();

rex.__proto__ === Dog.prototype; // true
```

4. Function Fundamentals

Q13. What are the different ways to declare a function in JavaScript?

Function declarations (hoisted fully), function expressions (not hoisted the same way), arrow functions (no own `this`/arguments), and the rarely-used Function constructor.

```
function declared() {}           // hoisted, callable before definition
const expr = function () {};    // not hoisted
const arrow = () => {};          // lexical this, no arguments object
```

Q14. What is function hoisting, and how does it differ for declarations vs. expressions?

Function **declarations** are hoisted with their full body, so they can be called before they appear in the code. Function **expressions** assigned to `let`/`const` are hoisted as uninitialized bindings — calling them early throws a `ReferenceError` (temporal dead zone).

```
greet(); // works -- 'Hi!'
function greet() { console.log('Hi!'); }

sayBye(); // ReferenceError: Cannot access before initialization
const sayBye = () => console.log('Bye!');
```

Q15. What are default parameters and rest parameters?

Default parameters supply a fallback value when an argument is undefined. Rest parameters (`...args`) collect any remaining arguments into a real array, replacing the old arguments object pattern.

```
function greet(name = 'friend', ...rest) {
  return `Hi ${name}! Extra args: ${rest.join(', ')}`;
}

greet(); // 'Hi friend! Extra args: '
greet('Amy', 'x', 'y'); // 'Hi Amy! Extra args: x, y'
```

Q16. What's the difference between a pure function and an impure function?

A pure function always returns the same output for the same input and causes no side effects (no mutating outside state, no I/O). Impure functions may depend on or modify external state, making them harder to test and reason about.

```
// Pure
function add(a, b) { return a + b; }

// Impure -- depends on & mutates external state
let total = 0;
function addToTotal(n) { total += n; return total; }
```

5. 'this', call/apply/bind & Arrow Functions

Q17. How is the value of 'this' determined in a regular function?

this is determined by **how a function is called**, not where it's defined. As a method, it's the object before the dot; called standalone, it's undefined in strict mode (or the global object otherwise); with new, it's the newly created instance.

```
const user = {
  name: 'Ana',
  greet() { return `Hi, I'm ${this.name}`; }
};

user.greet();           // 'Hi, I'm Ana' -- this = user
const fn = user.greet;
fn();                   // this is undefined -- breaks!
```

Q18. How do arrow functions handle 'this' differently?

Arrow functions don't have their own this. Instead, they capture this lexically from the surrounding scope at the time they're defined, which makes them useful for callbacks inside methods but unsuitable as object methods themselves.

```
const timer = {
  seconds: 0,
  start() {
    setInterval(() => {
      this.seconds++; // lexical this -- correctly refers to timer
    }, 1000);
  }
};
```

Q19. What's the difference between call(), apply(), and bind()?

All three let you explicitly set this for a function. call invokes immediately with arguments listed individually; apply invokes immediately with arguments as an array; bind returns a **new function** with this permanently bound, without calling it.

```
function introduce(greeting) { return `${greeting}, I'm ${this.name}`; }
const person = { name: 'Max' };

introduce.call(person, 'Hi');           // 'Hi, I'm Max'
introduce.apply(person, ['Hey']);       // 'Hey, I'm Max'
const bound = introduce.bind(person);
bound('Yo');                           // 'Yo, I'm Max'
```

6. Closures, Scope & Higher-Order Functions

Q20. What is a closure? Give a practical example.

A closure is a function that retains access to variables from its outer (enclosing) scope even after that outer function has finished executing. Closures are the mechanism behind private state, memoization, and function factories.

```
function makeCounter() {
  let count = 0;
  return () => ++count; // closes over 'count'
}

const counter = makeCounter();
counter(); // 1
counter(); // 2 -- count persists between calls
```

Q21. What's the classic closure-in-a-loop bug, and how do you fix it?

Using `var` in a loop with an async callback captures a single, shared binding — by the time the callback runs, the loop has finished and the variable holds its final value. Switching to `let` creates a new binding per iteration, fixing it.

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0); // logs 3, 3, 3
}

for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log(j), 0); // logs 0, 1, 2
}
```

Q22. What is a higher-order function?

A function that either takes one or more functions as arguments, returns a function, or both. Array methods like `map/filter` are higher-order functions, as are factories like `debounce` or `bind`.

```
function withLogging(fn) {
  return (...args) => {
    console.log('calling with', args);
    return fn(...args);
  };
}

const loggedAdd = withLogging((a, b) => a + b);
loggedAdd(2, 3); // logs args, returns 5
```

Q23. What is currying, and why is it useful?

Currying transforms a function that takes multiple arguments into a sequence of functions that each take a single argument. It's useful for creating specialized, reusable functions from a general one (partial application).

```
const multiply = a => b => a * b;

const double = multiply(2);
double(5); // 10
multiply(3)(4); // 12
```

7. Coding Challenges

Q24. Implement a simple deep clone function without using structuredClone.

```
function deepClone(value) {
  if (value === null || typeof value !== 'object') return value;
  if (Array.isArray(value)) return value.map(deepClone);
  const result = {};
  for (const key in value) {
    if (value.hasOwnProperty(key)) result[key] = deepClone(value[key]);
  }
  return result;
}
```

Q25. Implement your own version of Function.prototype.bind.

```
Function.prototype.myBind = function (thisArg, ...boundArgs) {
  const fn = this;
  return function (...callArgs) {
    return fn.apply(thisArg, [...boundArgs, ...callArgs]);
  };
};

function greet(greeting) { return `${greeting}, ${this.name}`; }
const bound = greet.myBind({ name: 'Zoe' });
bound('Hi'); // 'Hi, Zoe'
```

Q26. Write a debounce function.

Debouncing ensures a function only runs after a burst of calls has stopped for a given delay — commonly used for search-input or resize-event handlers.

```
function debounce(fn, delay) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}

const onResize = debounce(() => console.log('resized!'), 300);
window.addEventListener('resize', onResize);
```

Q27. Write a function that memoizes the result of another function.

```
function memoize(fn) {
  const cache = new Map();
  return (...args) => {
    const key = JSON.stringify(args);
    if (cache.has(key)) return cache.get(key);
    const result = fn(...args);
    cache.set(key, result);
    return result;
  };
}
```

```
const slowSquare = n => n ** 2;  
const fastSquare = memoize(slowSquare); // caches repeated calls
```

Q28. Implement a simple event emitter using closures.

```
function createEmitter() {  
  const listeners = {};  
  return {  
    on(event, cb) {  
      (listeners[event] ??= []).push(cb);  
    },  
    emit(event, ...args) {  
      (listeners[event] || []).forEach(cb => cb(...args));  
    }  
  };  
}  
  
const emitter = createEmitter();  
emitter.on('greet', name => console.log(`Hi, ${name}!`));  
emitter.emit('greet', 'Sam'); // 'Hi, Sam!'
```

Tip for interviews: for object/function questions, be ready to explain *why* JavaScript behaves a certain way (hoisting, the prototype chain, lexical scoping) — interviewers usually care more about your mental model than the exact syntax.